Here is the fully finalized, over-explained project architecture that integrates both the core "Obstacle Runner" mechanics and the novelty features to ensure your group gets maximum marks.

---

1. The Complete Feature List (Core + Novelty)

Core Engine & Environment

- **Infinite Scrolling Floor:** A grid made of GL_QUADS that continuously translates towards the camera and snaps back to simulate endless forward momentum.
- **Three-Lane System:** Fixed X-axis coordinates ($x = -100, 0, 100$) restricting player movement.
- **HUD (Heads-Up Display):** Real-time text using draw_text showing Score (based on time survived) and Health/Lives.

Player Mechanics

- **Run (Default):** Player is a glutSolidCube moving forward.
- **Smooth Lane Switching:** Interpolating the X-coordinate smoothly rather than instantly teleporting.
- **Jump:** Parabolic Y-axis translation using kinematic math.
- **Duck/Roll:** Scaling the Y-axis of the player down (glScalef) to dodge high obstacles.
- ⭐ **NOVELTY - Gravity Inversion:** The player can pick up a specific item that instantly flips their gravity, forcing them to run upside-down on the "ceiling" of the arcade grid.

Obstacles & Physics

- **Procedural Generation:** Obstacles spawn randomly at the far end of the Z-axis in one of the three lanes.
- **AABB Collision Detection:** Mathematical checks to determine if the player's 3D boundaries intersect with an obstacle.
- ⭐ **NOVELTY - Morphing Obstacles:** Obstacles actively change shape (e.g., smoothly transitioning from a gluCylinder to a gluSphere) as they cross a specific Z-axis threshold near the player.

- ⭐ **NOVELTY - Dynamic Warp Speed:** As the score increases, the global game speed increases, and the camera's Field of View (fovY) dynamically widens, creating a visual "warp" effect.

---

2. Task Distribution (Equal Complexity)

Member 1: The World & Camera Architect

- **Core Task:** Implement the scrolling floor illusion and the HUD.
- **Novelty Task:** Implement the **Dynamic Warp Speed**. You will tie a global speed_multiplier variable to the camera's fovY inside setupCamera().

Member 2: The Player Controller & Physics

- **Core Task:** Implement the player cube, smooth lane switching, jumping, and rolling states.
- **Novelty Task:** Implement **Gravity Inversion**. You must handle the logic to flip the player's Y-axis bounds, adjust the jump math to work upside down, and render a ceiling grid for them to run on.

Member 3: The Obstacle Engine & Collision Systems

- **Core Task:** Create the obstacle manager (spawning, translating, and deleting shapes) and write the AABB collision math.
- **Novelty Task:** Implement **Morphing Obstacles**. You must write logic that checks the obstacle's Z-distance from the camera and swaps its OpenGL drawing function and bounding box mid-run.

---

3. GitHub & Task Completion Sequence

Because you are forced to use a **single .py file**, you must be extremely disciplined to avoid GitHub merge conflicts.

The "Zoned Coding" Strategy

1. **Initialize:** Member 1 pushes the base template to the main branch.

2. **Zoning:** Member 1 adds massive comment blocks to the file to create strict boundaries. No one is allowed to write functions outside their designated zone.
    - # === GLOBAL VARIABLES ===
    - # === MEMBER 1: WORLD & CAMERA ===
    - # === MEMBER 2: PLAYER ===
    - # === MEMBER 3: OBSTACLES ===
    - # === MAIN GAME LOOP ===
3. **Global Meeting:** Before coding, all three of you must agree on the global variables (e.g., player_x, player_y, player_z, game_speed, active_obstacles list) and put them in the global zone.

The Execution Sequence

- **Phase 1: Environment (Member 1).** Member 1 creates a branch, builds the scrolling floor and camera setup, and merges into main. The game now looks like it is moving forward.
- **Phase 2: The Player (Member 2).** Member 2 pulls the updated main, creates a branch, and adds the player cube and movement controls. Merges into main. The game now has a moving floor and a controllable cube.
- **Phase 3: The Threat (Member 3).** Member 3 pulls main, creates a branch, and adds the obstacles and collision detection. Merges into main. The core game is now playable.
- **Phase 4: Novelty Integration (Parallel).** All members branch off again to add their specific novelty features. Because the core functions are already built in separate zones, these additions will not conflict.

---

4. Gemini Prompts for Group Members

Save the following blocks into three text files and distribute them to your team.

File 1: Prompt_Member1_World.txt
Plaintext
You are an expert in legacy OpenGL using Python and GLUT. I am building a 3D "Subway Surfers" style infinite runner for a university project. We are strictly limited to basic shapes (glutSolidCube, gluCylinder, gluSphere) and transformations (glTranslate, glRotate, glScale). NO shaders, NO textures. I am Member 1, responsible for the World, Camera, HUD, and a Dynamic FOV novelty feature.

From this point on, every code snippet you give me must be in Python.

My specific tasks are:
1. Ensure `GL_DEPTH_TEST` is enabled in the display initialization so 3D objects render correctly.
2. Create an infinite scrolling floor. I need to draw a grid using `GL_QUADS` that translates along the Z-axis over time and snaps back to its starting position seamlessly to create the illusion of forward movement.
3. Implement a HUD using the existing `draw_text` function to show a Score that increments based on time.
4. NOVELTY FEATURE - Dynamic Warp Speed: Update the `setupCamera()` function. I need logic where as the global `game_speed` increases over time, the camera's `fovY` gently widens from 120 up to a maximum of 150, creating a "warp speed" visual effect.

Please provide the Python code blocks for these specific features. Structure the code so it is modular and can be easily inserted into a specific "Environment" section of a larger single file.

File 2: Prompt_Member2_Player.txt
Plaintext
You are an expert in legacy OpenGL using Python and GLUT. I am building a 3D "Subway Surfers" style infinite runner for a university project. We are strictly limited to basic shapes (glutSolidCube, gluCylinder, gluSphere) and transformations (glTranslate, glRotate, glScale). NO shaders, NO textures. I am Member 2, responsible for the Player Controller, Physics, and a Gravity Inversion novelty feature.

From this point on, every code snippet you give me must be in Python.

My specific tasks are:
1. Draw the player (a `glutSolidCube`).
2. Handle Lane Switching: The player can be in 3 lanes (e.g., X = -100, 0, 100). When left/right arrows are pressed, smoothly interpolate the player's X coordinate to the new lane over a few frames, rather than teleporting instantly.
3. Handle Jumping and Rolling: Implement a time-based parabolic jump for the Y coordinate, and a roll state that uses `glScalef` to squash the cube.

4. NOVELTY FEATURE - Gravity Inversion: I need a state variable `gravity_inverted`. When true, the player instantly translates to the "ceiling" (e.g., Y = 400). The jump math must be reversed so the player jumps "down" toward the floor and falls back "up" to the ceiling.

Please provide the Python variables (state machine) and functions needed to handle these mechanics. Keep it modular so I can drop it into my team's single main file.

File 3: Prompt_Member3_Obstacles.txt
Plaintext
You are an expert in legacy OpenGL using Python and GLUT. I am building a 3D "Subway Surfers" style infinite runner for a university project. We are strictly limited to basic shapes (glutSolidCube, gluCylinder, gluSphere) and transformations (glTranslate, glRotate, glScale). NO shaders, NO textures. I am Member 3, responsible for Obstacle Spawning, Collision Detection, and a Morphing Obstacle novelty feature.

From this point on, every code snippet you give me must be in Python.

My specific tasks are:
1. Create an obstacle manager (a list of dictionaries/objects). Each obstacle needs a type, a lane (X pos), a Z pos, and an active state.
2. Write an update function that spawns obstacles randomly far down the Z-axis, moves them toward Z=0 every frame based on `game_speed`, and removes them when they pass the camera.
3. Implement 3D Axis-Aligned Bounding Box (AABB) collision math between the player cube and the obstacles.
4. NOVELTY FEATURE - Morphing Obstacles: I need logic inside the obstacle rendering loop that checks the obstacle's Z-distance from the camera. If it gets within a certain threshold, the obstacle must change its draw type (e.g., transitioning from a `gluCylinder` to a `gluSphere`) and update its bounding box accordingly to trick the player.

Please provide the Python code for this obstacle manager, the AABB math, and the morphing logic. Keep it modular so it won't conflict with my teammates' player code.

GROUP TASK SEQUENCE

Here is the step-by-step task completion sequence for your group, designed specifically to prevent GitHub merge conflicts since you are all working within a single .py file.

The "Zoned Coding" GitHub Sequence

Step 0: Initialization & Zoning (Member 1)

- **Action:** Member 1 sets up the GitHub repository with the base 3D_OpenGL_Intro (1).py template.
- **The Crucial Step:** Member 1 adds massive comment blocks to the file to create strict boundaries for everyone (e.g., # === MEMBER 2: PLAYER ===).
- **Action:** Member 1 pushes this to the main branch. *No one is allowed to write functions outside their designated zone.*

Phase 1: The Environment (Member 1)

- **Action:** Member 1 creates a new branch (feature-world).
- **Task:** Code the scrolling floor grid and the camera setup inside the "World" zone.
- **Merge:** Member 1 merges their branch into main.
- *Status: The game now has an endless scrolling floor and a camera.*

Phase 2: The Player (Member 2)

- **Action:** Member 2 pulls the updated main branch to their local machine, then creates a new branch (feature-player).
- **Task:** Code the player cube, smooth lane switching, jumping, and rolling states inside the "Player" zone.
- **Merge:** Member 2 tests their player on the moving floor, then merges their branch into main.
- *Status: The game now has a moving floor and a controllable player cube.*

Phase 3: The Threat (Member 3)

- **Action:** Member 3 pulls the updated main branch to their local machine, then creates a new branch (feature-obstacles).
- **Task:** Code the obstacle manager (spawning/translating) and the AABB collision detection inside the "Obstacles" zone.
- **Merge:** Member 3 tests that the obstacles actually hit the player cube, then merges their branch into main.
- *Status: The core game is now fully playable.*

Phase 4: Novelty Integration (All Members in Parallel)

- **Action:** Now that the core engine works and the code is safely segregated into zones, all three members can safely pull main and branch off simultaneously.
- **Tasks:** * Member 1 codes **Dynamic Warp Speed**.
    - Member 2 codes **Gravity Inversion**.
    - Member 3 codes **Morphing Obstacles**.
- **Merge:** Because everyone is working in their own pre-defined comment zones, you can merge these features back into main in any order with minimal risk of conflicts.